00 — بنية التطبيق وترابط الطبقات

مشروع رزمة · ملف شرح للفريق

الغرض

رزمة تطبيق تعلّم **يعمل دون إنترنت أولًا**. كل ما يراه الطالب يأتي من جهازه: ملفه، تقدّمه، ومحتوى الدروس. هذا الملف يشرح كيف تترابط أجزاء الكود حتى يعرف أي عضو في الفريق أين يضع الكود الجديد وأين يبحث عن أي شيء.

فهرس بقية الملفات: `README.md` في هذا المجلد.

الخريطة العامة

```
lib/
  main.dart                    يشغّل التطبيق داخل ProviderScope
  app/
    rizma_app.dart             MaterialApp.router + العربية RTL + الثيم
    router.dart                كل المسارات (go_router) وقواعد التحويل
    routes.dart                أسماء المسارات (ثوابت) — منفصلة كي لا تعتمد الشاشات على الراوتر
  core/                        كل ما تشترك فيه أكثر من ميزة
    db/                        قاعدة البيانات (Drift): جداول + DAOs + مزوّد القاعدة
    theme/                     الألوان والأنماط والثيم وألوان شخصية الطالب
    widgets/                   مكوّنات مشتركة (11 انظر _shared_widgets.md)
    utils/                     دوال صرفة بلا واجهة (رمز الطالب، تجزئة الرمز السري، السلسلة…)
    strings/ar.dart            كل النصوص الظاهرة للمستخدم
  features/                    ميزة = مجلد (شاشات + مزوّدات + مستودع)
    launch/                    قرار «إلى أين نذهب عند فتح التطبيق؟»
    onboarding/                الترحيب + إنشاء الملف (الخطوتان)
    profiles/                  الملفات: المستودع، اختيار الملف، الرمز السري
    home/                      الرئيسية: الحالة المشتقة + الودجات + أدوات التطوير
    catalog/                   كتالوج المحتوى (JSON) ونماذجه
assets/
  content/catalog_grade5.json  هيكل المحتوى (مواد ← وحدات ← دروس ← مفاهيم)
  fonts/Cairo-*.ttf            الخط داخل التطبيق (يعمل دون إنترنت)
  icons/ images/               أصول SVG من Figma
web/                           ملفات التشغيل على Chrome (تتضمن sqlite3.wasm و drift_worker.js)
design_spec/                   مواصفات الشاشات 25–29 من فريق التصميم (مرجع فقط)
```

الطبقات واتجاه الاعتماد

الاعتماد يسير **من الأعلى إلى الأسفل فقط**:

```
الشاشة (Widget)
   | تقرأ وتنادي
   ▼
المزوّد (Riverpod Provider / Notifier)
   | يستخدم
   ▼
المستودع (Repository)  ← يضع قواعد العمل (التحقق، القفل، توليد الرمز…)
   | يستخدم
   ▼
DAO (Drift)            ← استعلامات SQL مكتوبة بـ Dart
   |
   ▼
SQLite على الجهاز
```

- **الشاشة** لا تعرف شيئًا عن SQL ولا عن تجزئة الرمز السري. تعرض حالة وتنادي دوالّ.
- **المستودع** هو مكان القواعد. مثال: `ProfileRepository.verifyPin` يقرر هل الرمز صحيح وهل الملف مقفل.
- **DAO** يقرأ ويكتب فقط، بلا قواعد.
- **core/utils** دوال بلا اعتماد على Flutter (تُختبر كـ Dart صرف).
- **الحالة المشتقة** (مثل `HomeState`) دالة صرفة تأخذ صفوف القاعدة والكتالوج وتعيد «ما يجب عرضه». لا تلمس القاعدة ولا Flutter، لذلك تُختبر بدون واجهة.

كيف تصل البيانات للشاشة (Riverpod)

مثال الرئيسية، من الأسفل للأعلى:

```
appDatabaseProvider  ──►  profileRepositoryProvider
        │                          │
        │                 activeProfileIdProvider (AsyncNotifier: الملف النشط المحفوظ)
        │                          │
        ├─► lessonProgressProvider(profileId)  ─┐
        ├─► attemptsProvider(profileId)         ├─► homeStateProvider ──► HomeScreen
        ├─► masteryProvider(profileId)          │
        └── catalogProvider (من JSON) ──────────┘
            activeProfileProvider (Stream) ─┘
```

- `StreamProvider` على استعلامات Drift: عند تغيّر بيانات الجدول يصدر `Stream` قيمة جديدة فتُعاد الحالة والواجهة تلقائيًا، بلا `setState` يدوي.
- `homeStateProvider` يجمع المصادر في `AsyncValue`: إن كان أحدها يحمّل فالنتيجة `loading`، وإن فشل أحدها `error`.
- في الاختبارات نستبدل `appDatabaseProvider` بقاعدة في الذاكرة و`catalogRepositoryProvider` بنسخة تقرأ النص المحمَّل مسبقًا (انظر `12_testing.md`).

التنقل

`go_router` في `app/router.dart`. القاعدة: الشاشة تنتقل بـ `context.go/push(AppRoutes.xxx)` ولا تستورد شاشة أخرى مباشرة. تفاصيل المسارات وقواعد الإقلاع في `06_launch_and_navigation.md`.

العربية والاتجاه (RTL)

- `locale: ar` في `MaterialApp` يقلب الاتجاه تلقائيًا: **أول عنصر في `Row` يظهر على اليمين**.
- نستخدم `EdgeInsetsDirectional` و`AlignmentDirectional` بدل `left/right`.

- الأرقام غربية (0–9) كما في التصميم.
- لوحة أرقام الرمز السري LTR عمدًا (1 2 3 / 4 5 6 / 7 8 9) كما في التصميم.
- صيغ الجمع العربية (نجمة/نجمتان/3 نجوم/11 نجمة) في `ArStrings.stars` و `streakDays`.

الثيم والخط والنصوص

- `AppColors` قيم Figma حرفيًا. `AppTextStyles` أنماط Cairo (الاسم يحمل الحجم، مثل `heading22`).
- ألوان شخصية الطالب الخمسة في `core/theme/profile_palette.dart`.
- الخط داخل التطبيق (`assets/fonts`) ومعرَّف في `pubspec.yaml` بأوزان 400/600/700/800. لا اعتماد على الإنترنت.
- **كل نص ظاهر** يُكتب في `core/strings/ar.dart`، لا داخل الشاشات (الاستثناء الوحيد: نصوص أدوات التطوير التي لا تظهر إلا في debug).

قاعدة: أرقام التصميم أمثلة

أي رقم أو اسم في Figma (24، 75%، نجمة، «أنس»...) مثال لشكل الشاشة. الكود لا ينسخه: القيم تُحسب من قاعدة البيانات والكتالوج. لا يوجد ملف بيانات تجريبية في المشروع.

التشغيل على Chrome (للتطوير)

الهدف النهائي هاتف أندرويد، لكن يمكن التجريب على Chrome:

```
flutter run -d chrome --web-port=5000
```

- ثبّت المنفذ (`--web-port`)؛ بياناتك تُحفظ في IndexedDB **لكل منفذ**، فمنفذ عشوائي جديد في كل تشغيل يعني قاعدة فارغة.
- Drift على الويب يحتاج ملفين في مجلد `web/` : `sqlite3.wasm` و `drift_worker.js`، وهما منسوخان من إصدارات Drift الرسمية. **يجب أن يطابقا إصدارات الحزم** في `pubspec.lock` (الآن: `sqlite3` 2.9.4 و`drift` 2.31.0). عند ترقية أي حزمة نزّل الملفين من الإصدار المطابق:
  - `https://github.com/simolus3/sqlite3.dart/releases` (ملف `sqlite3.wasm`)
  - `https://github.com/simolus3/drift/releases` (ملف `drift_worker.js`)
- بدون هذين الملفين أو بدون الإعداد `web:` في `AppDatabase.onDevice()` يفشل إنشاء الملف برسالة «تعذّر إنشاء الملف». في وضع debug يظهر السبب الحقيقي تحت الرسالة وفي سجل المتصفح.

قرارات تقنية تستحق المعرفة

القرار	السبب
`drift` 2.31 و`drift_flutter` 0.2.x و`flutter_riverpod` 2.6.1	Flutter 3.38.3 يثبّت `meta 1.17`، وأحدث `drift_dev` و `riverpod 3` يتعارضان معه. الترقية الكاملة تتطلب ترقية Flutter.
ملفات `*.g.dart` داخل Git	حتى يعمل المشروع بعد النسخ دون تشغيل `build_runner`. تُولَّد بـ `dart run build_runner build`.
`minSdk = 24`	هو الحد الأدنى الذي يدعمه Flutter 3.38 رسميًا.
`google_fonts` محذوف	يجلب الخط من الإنترنت وهذا يخالف مبدأ «دون إنترنت أولًا».

القرار	السبب
بطاقات «الرئيسية» مبنية على `HomeState` لا على القاعدة مباشرة	يفصل «ماذا نعرض» عن «كيف نرسم»، ويُختبر بلا واجهة.

الشاشات الضيقة والواسعة

- **ضيقة (320) وخط مكبّر:** لا نكتب عروضًا ثابتة تفيض. القاعدة: صف قد لا يتسع ← `Wrap` (ينزل العنصر لسطر جديد) أو `FittedBox(scaleDown)` (يتقلّص قليلًا) أو `Flexible/Expanded`. يحرس هذا `test/layout_stress_test.dart`.
- **واسعة (كروم):** `MaterialApp.router(builder: ...)` في `rizma_app.dart` يلفّ كل الشاشات في `ConstrainedBox(maxWidth: 480)` في أعلى الوسط على خلفية رمادية. لأن `builder` فوق `Navigator` فالأوراق السفلية والرسائل تتقيّد بالعرض نفسه.

الحركة واللمس

كل انتقال شاشة بحركة قصيرة (`appPage`)، والمحتوى يظهر تباعًا (`.fadeIn(n)`)، والضغط يصغّر العنصر قليلًا (`.pressable()`) ويهتزّ الجهاز نقرة خفيفة (`Haptics`). التفاصيل والأسباب في `15_motion_and_delete.md`.